

老子软考，扫码下载



老子
软考

老子软考
软考用老子

软件设计师-2016年下半年-选择题

第1题：在程序运行过程中，CPU需要将指令从内存中取出并加以分析和执行。CPU依据（ ）来区分在内存中以二进制编码形式存放的指令和数据。

- A. 指令周期的不同阶段
- B. 指令和数据的寻址方式
- C. 指令操作码的译码结果
- D. 指令和数据所在的存储单元

第2题：计算机在一个指令周期的过程中，为从内存读取指令操作码，首先要将（ ）的内容送到地址总线上。

- A. 指令寄存器（IR）
- B. 通用寄存器（GR）
- C. 程序计数器（PC）
- D. 状态寄存器（PSW）

第3题：设16位浮点数，其中阶符1位、阶码值6位、数符1位、尾数8位。若阶码用移码表示，尾数用补码表示，则该浮点数所能表示的数值范围是（ ）。

- A. $-2^{64} \sim (1-2^{-8}) 2^{64}$
- B. $-2^{63} \sim (1-2^{-8}) 2^{63}$
- C. $-2^{64} \sim (1-2^{-(1-2^{-8})}) 2^{64} \sim (1-2^{-8}) 2^{64}$
- D. $-(1-2^{-8}) 2^{63} \sim (1-2^{-8}) 2^{63}$

第4题：已知数据信息为16位，最少应附加（ ）位校验位，以实现海明码纠错。

- A. 3
- B. 4
- C. 5
- D. 6

第5题：将一条指令的执行过程分解为取指、分析和执行三步，按照流水方式执行，若取指时间 $t_{\text{取指}}=4\Delta t$ 、分析时间 $t_{\text{分析}}=2\Delta t$ 、执行时间 $t_{\text{执行}}=3\Delta t$ ，则执行完100条指令，需要的时间为（ ） Δt 。

- A. 200
- B. 300
- C. 400
- D. 405

第6题：以下关于Cache与主存间地址映射的叙述中，正确的是（ ）。

- A. 操作系统负责管理Cache与主存之间的地址映射
- B. 程序员需要通过编程来处理Cache与主存之间的地址映射
- C. 应用软件对Cache与主存之间的地址映射进行调度
- D. 由硬件自动完成Cache与主存之间的地址映射

第7题：可用于数字签名的算法是（ ）。

- A. RSA
- B. IDEA
- C. RC4
- D. MD5

第8题：（ ）不是数字签名的作用。

- A. 接收者可验证消息来源的真实性
- B. 发送者无法否认发送过该消息
- C. 接收者无法伪造或篡改消息
- D. 可验证接收者合法性

第9题：在网络设计和实施过程中要采取多种安全措施，其中（ ）是针对系统安全需求的措施。

- A. 设备防雷击
- B. 入侵检测
- C. 漏洞发现与补丁管理
- D. 流量控制

第10题：（ ）的保护期限是可以延长的。

- A. 专利权
- B. 商标权
- C. 著作权

D. 商业秘密权

第11题：甲公司软件设计师完成了一项涉及计算机程序的发明。之后，乙公司软件设计师也完成了与甲公司软件设计师相同的涉及计算机程序的发明。甲、乙公司于同一天向专利局申请发明专利。此情形下，（ ）是专利权申请人。

- A. 甲公司
- B. 甲、乙两公司
- C. 乙公司
- D. 由甲、乙公司协商确定的公司

第12题：甲、乙两厂生产的产品类似，且产品都使用“B”商标。两厂于同一天向商标局申请商标注册，且申请注册前两厂均未使用“B”商标。此情形下，（ ）能核准注册。

- A. 甲厂
- B. 由甲、乙厂抽签确定的厂
- C. 乙厂
- D. 甲、乙两厂

第13题：在FM方式的数字音乐合成器中，改变数字载波频率可以改变乐音的（13），改变它的信号幅度可以改变乐音的（14）。

- A. 音调
- B. 音色
- C. 音高
- D. 音质

第14题：在FM方式的数字音乐合成器中，改变数字载波频率可以改变乐音的（13），改变它的信号幅度可以改变乐音的（14）。

- A. 音调
- B. 音域
- C. 音高
- D. 带宽

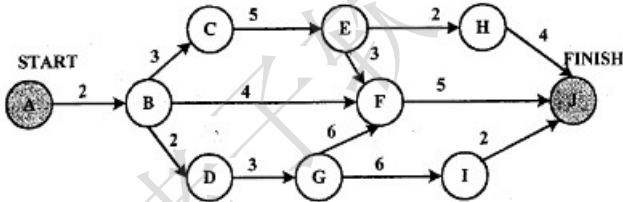
第15题：结构化开发方法中，（ ）主要包含对数据结构和算法的设计。

- A. 体系结构设计
- B. 数据设计
- C. 接口设计
- D. 过程设计

第16题：在敏捷过程的开发方法中，（ ）使用了迭代的方法，其中，把每段时间（30天）一次的迭代称为一个“冲刺”，并按需求的优先级别来实现产品，多个自组织和自治的小组并行地递增实现产品。

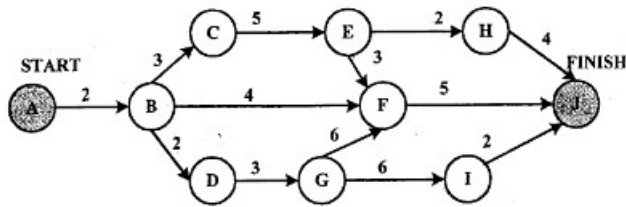
- A. 极限编程XP
- B. 水晶法
- C. 并列争球法
- D. 自适应软件开发

第17题：某软件项目的活动图如下图所示，其中顶点表示项目里程碑，连接顶点的边表示包含的活动，边上的数字表示相应活动的持续时间（天），则完成该项目的最少时间为（17）天。活动BC和BF最多可以晚开始（18）天而不会影响整个项目的进度。



- A. 11
- B. 15
- C. 16
- D. 18

第18题：某软件项目的活动图如下图所示，其中顶点表示项目里程碑，连接顶点的边表示包含的活动，边上的数字表示相应活动的持续时间（天），则完成该项目的最少时间为（17）天。活动BC和BF最多可以晚开始（18）天而不会影响整个项目的进度。



- A. 0和7
- B. 0和11
- C. 2和7
- D. 2和11

第19题：成本估算时，（ ）方法以规模作为成本的主要因素，考虑多个成本驱动因子。该方法包括三个阶段模型，即应用组装模型、早期设计阶段模型和体系结构阶段模型。

- A. 专家估算
- B. Wolverton
- C. COCOMO
- D. COCOMO II

第20题：逻辑表达式求值时常采用短路计算方式。“&&”、“||”、“!”分别表示逻辑与、或、非运算，“&&”、“||”为左结合，“!”为右结合，优先级从高到低为“!”、“&&”、“||”。对逻辑表达式“x&&(y||z)”进行短路计算方式求值时，（ ）。

- A. x为真，则整个表达式的值即为真，不需要计算y和z的值
- B. x为假，则整个表达式的值即为假，不需要计算y和z的值
- C. x为真，再根据z的值决定是否需要计算y的值
- D. x为假，再根据y的值决定是否需要计算z的值

第21题：常用的函数参数传递方式有传值与传引用两种。（ ）。

- A. 在传值方式下，形参与实参之间互相传值
- B. 在传值方式下，实参不能是变量
- C. 在传引用方式下，修改形参实质上改变了实参的值。
- D. 在传引用方式下，实参可以是任意的变量和表达式。

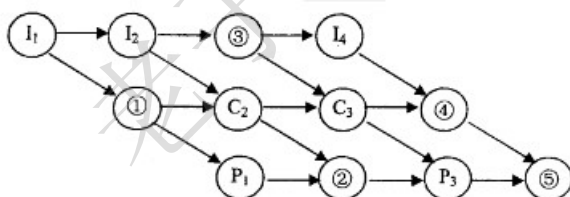
第22题：二维数组a[1..N, 1..N]可以按行存储或按列存储。对于数组元素a[i,j] (1≤i,j≤N)，当（ ）时，在按行和按列两种存储方式下，其偏移量相同。

- A. i=j
- B. i-j
- C. i+j
- D. i-j

第23题：实时操作系统主要用于有实时要求的过程控制等领域。实时系统对于来自外部的事件必须在（ ）。

- A. 一个时间片内进行处理
- B. 一个周转时间内进行处理
- C. 一个机器周期内进行处理
- D. 被控对象规定的时间内做出及时响应并对其进行处理

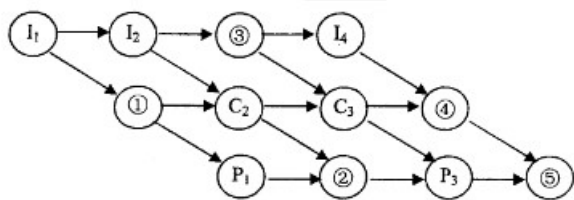
第24题：假设某计算机系统中只有一个CPU、一台输入设备和一台输出设备，若系统中有四个作业T1、T2、T3和T4，系统采用优先级调度，且T1的优先级>T2的优先级>T3的优先级>T4的优先级。每个作业Ti具有三个程序段：输入Ii、计算Ci和输出Pi (i=1, 2, 3, 4)，其执行顺序为Ii→Ci→Pi。这四个作业各程序段并发执行的前驱图如下所示。图中①、②分别为（24），③、④、⑤分别为（25）。



- A. I2、P2
- B. I2、C2
- C. C1、P2
- D. C1、P3

第25题：假设某计算机系统中只有一个CPU、一台输入设备和一台输出设备，若系统中有四个作业T1、T2、T3和T4，系统采用优先级调度，且T1的优先级>T2的优先级>T3的优先级>T4的优先级。每个作业Ti具有三个程序段：输入Ii、计算Ci和输出

Pi (i=1, 2, 3, 4) , 其执行顺序为Ii→Ci→Pi。这四个作业各程序段并发执行的前驱图如下所示。图中①、②分别为(24) , ③、④、⑤分别为(25) 。



- A. C2、C4、P4
- B. I2、I3、C4
- C. I3、P3、P4
- D. I3、C4、P4

第26题：假设段页式存储管理系统中的地址结构如下图所示，则系统（ ）。

3 1	24 23	13 12	0
段 号	页 号	页内地址	

- A. 最多可有256个段，每个段的大小均为2048个页，页的大小为8K
- B. 最多可有256个段，每个段最大允许有2048个页，页的大小为8K
- C. 最多可有512个段，每个段的大小均为1024个页，页的大小为4K
- D. 最多可有512个段，每个段最大允许有1024个页，页的大小为4K

第27题：假设系统中有n个进程共享3台扫描仪，并采用PV操作实现进程同步与互斥。若系统信号量S的当前值为-1，进程P1、P2又分别执行了1次P（S）操作，那么信号量S的值应为（ ）。

- A. 3
- B. -3
- C. 1
- D. -1

第28题：某字长为32位的计算机的文件管理系统采用位示图（bitmap）记录磁盘的使用情况。若磁盘的容量为300GB，物理块的大小为1MB，那么位示图的大小为（ ）个字。

- A. 1200
- B. 3200
- C. 6400
- D. 9600

第29题：某开发小组欲为一公司开发一个产品控制软件，监控产品的生产和销售过程，从购买各种材料开始，到产品的加工和销售进行全程跟踪。购买材料的流程、产品的加工过程以及销售过程可能会发生变化。该软件的开发最不宜采用（29）模型，主要是因为这种模型（30）。

- A. 瀑布
- B. 原型
- C. 增量
- D. 喷泉

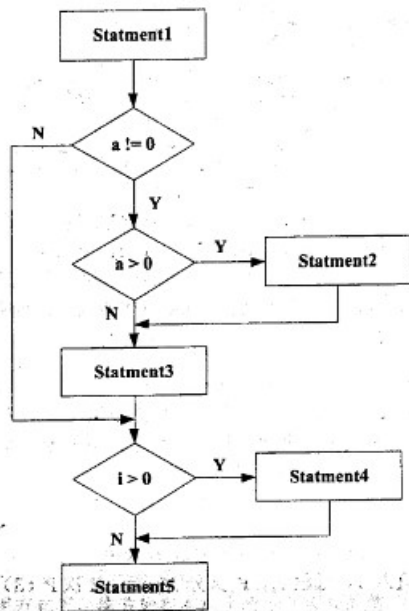
第30题：某开发小组欲为一公司开发一个产品控制软件，监控产品的生产和销售过程，从购买各种材料开始，到产品的加工和销售进行全程跟踪。购买材料的流程、产品的加工过程以及销售过程可能会发生变化。该软件的开发最不宜采用（29）模型，主要是因为这种模型（30）。

- A. 不能解决风险
- B. 不能快速提交软件
- C. 难以适应变化的需求
- D. 不能理解用户的需求

第31题：（ ）不属于软件质量特性中的可移植性。

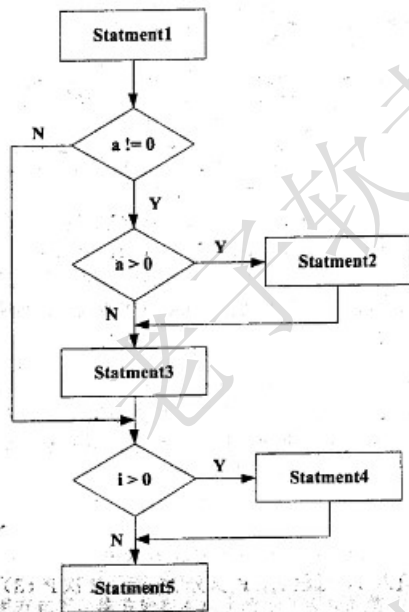
- A. 适应性
- B. 易安装性
- C. 易替换性
- D. 易理解性

第32题：对下图所示流程图采用白盒测试方法进行测试，若要满足路径覆盖，则至少需要（32）个测试用例。采用McCabe度量法计算该程序的环路复杂性为（33）。



- A. 3
- B. 4
- C. 6
- D. 8

第33题：对下图所示流程图采用白盒测试方法进行测试，若要满足路径覆盖，则至少需要（32）个测试用例。采用McCabe度量法计算该程序的环路复杂性为（33）。



- A. 1
- B. 2
- C. 3
- D. 4

第34题：计算机系统的（ ）可以用 $MTBF / (1 + MTBF)$ 来度量，其中MTBF为平均失效间隔时间。

- A. 可靠性
- B. 可用性
- C. 可维护性
- D. 健壮性

第35题：以下关于软件测试的叙述中，不正确的是（ ）。

- A. 在设计测试用例时应考虑输入数据和预期输出结果
- B. 软件测试的目的是证明软件的正确性
- C. 在设计测试用例时，应该包括合理的输入条件
- D. 在设计测试用例时，应该包括不合理的输入条件

第36题：某模块中有两个处理A和B，分别对数据结构X写数据和读数据，则该模块的内聚类型为（ ）内聚。

- A. 逻辑

- B. 过程
- C. 通信
- D. 内容

第37题：在面向对象方法中，不同对象收到同一消息可以产生完全不同的结果，这一现象称为（ ）。在使用时，用户可以发送一个通用的消息，而实现的细节则由接收对象自行决定。

- A. 接口
- B. 继承
- C. 覆盖
- D. 多态

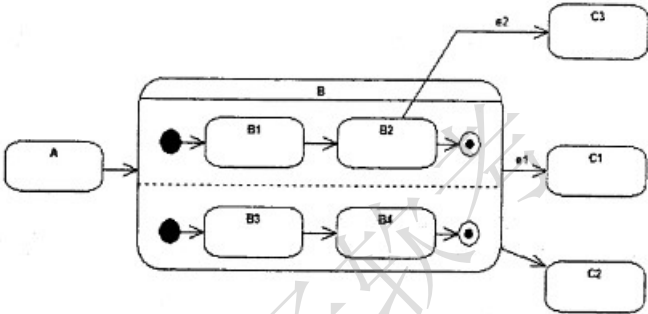
第38题：在面向对象方法中，支持多态的是（ ）。

- A. 静态分配
- B. 动态分配
- C. 静态类型
- D. 动态绑定

第39题：面向对象分析的目的是为了获得对应用问题的理解，其主要活动不包括（ ）。

- A. 认定并组织对象
- B. 描述对象间的相互作用
- C. 面向对象程序设计
- D. 确定基于对象的操作

第40题：如下所示的UML状态图中，（ ）时，不一定会离开状态B。

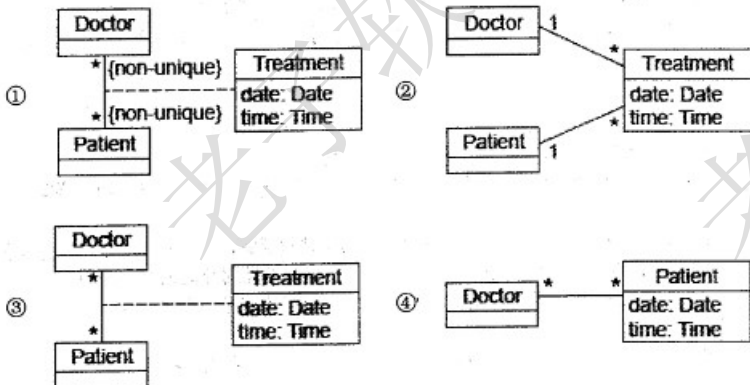


- A. 状态B中的两个结束状态均达到
- B. 在当前状态为B2时，事件e2发生
- C. 事件e2发生
- D. 事件e1发生

第41题：以下关于UML状态图中转换（transition）的叙述中，不正确的是（ ）。

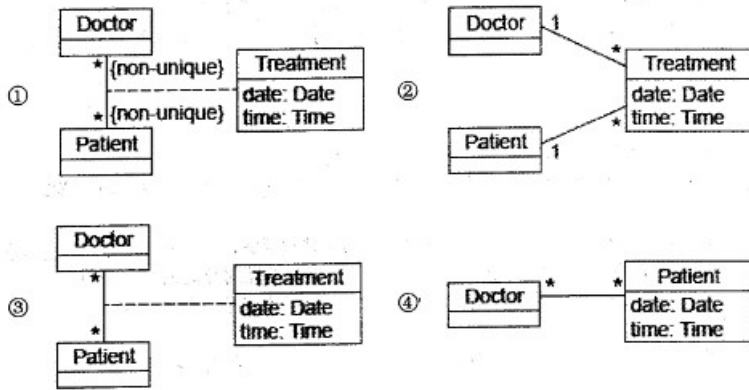
- A. 活动可以在转换时执行也可以在状态内执行
- B. 监护条件只有在相应的事件发生时才进行检查
- C. 一个转换可以有事件触发器、监护条件和一个状态
- D. 事件触发转换

第42题：下图①②③④所示是UML（42）。现有场景：一名医生（Doctor）可以治疗多位病人（Patient），一位病人可以由多名医生治疗，一名医生可能多次治疗同一位病人。要记录哪名医生治疗哪位病人时，需要存储治疗（Treatment）的日期和时间。以下①②③④图中（43）。是描述此场景的模型。



- A. 用例图
- B. 对象图
- C. 类图
- D. 协作图

第43题：下图①②③④所示是UML（42）。现有场景：一名医生（Doctor）可以治疗多位病人（Patient），一位病人可以由多名医生治疗，一名医生可能多次治疗同一位病人。要记录哪名医生治疗哪位病人时，需要存储治疗（Treatment）的日期和时间。以下①②③④图中（43）。是描述此场景的模型。



- A. ①
- B. ②
- C. ③
- D. ④

第44题：（44）模式定义一系列的算法，把它们一个个封装起来，并且使它们可以相互替换，使得算法可以独立于使用它们的客户而变化。以下（45）情况适合选用该模式。

- ①一个客户需要使用一组相关对象
- ②一个对象的改变需要改变其它对象
- ③需要使用一个算法的不同变体
- ④许多相关的类仅仅是行为有异

- A. 命令（Command）
- B. 责任链（Chain of Responsibility）
- C. 观察者（Observer）
- D. 策略（Strategy）

第45题：（44）模式定义一系列的算法，把它们一个个封装起来，并且使它们可以相互替换，使得算法可以独立于使用它们的客户而变化。以下（45）情况适合选用该模式。

- ①一个客户需要使用一组相关对象
- ②一个对象的改变需要改变其它对象
- ③需要使用一个算法的不同变体
- ④许多相关的类仅仅是行为有异

- A. ①②
- B. ②③
- C. ③④
- D. ①④

第46题：（46）模式将一个复杂对象的构建与其表示分离，使得同样的构建过程可以创建不同的表示。以下（47）情况适合选用该模式。

- ①抽象复杂对象的构建步骤
- ②基于构建过程的具体实现构建复杂对象的不同表示
- ③一个类仅有一个实例
- ④一个类的实例只能有几个不同状态组合中的一种

- A. 生成器（Builder）
- B. 工厂方法（Factory Method）
- C. 原型（Prototype）
- D. 单例（Singleton）

第47题：（46）模式将一个复杂对象的构建与其表示分离，使得同样的构建过程可以创建不同的表示。以下（47）情况适合选用该模式。

- ①抽象复杂对象的构建步骤
- ②基于构建过程的具体实现构建复杂对象的不同表示
- ③一个类仅有一个实例
- ④一个类的实例只能有几个不同状态组合中的一种

- A. ①②
- B. ②③
- C. ③④
- D. ①④

第48题：由字符a、b构成的字符串中，若每个a后至少跟一个b，则该字符串集合可用正规式表示为（ ）。

- A. $(b|ab)^*$
- B. $(ab^*)^*$
- C. $(a^*b^*)^*$
- D. $(a|b)^*$

第49题：乔姆斯基（Chomsky）将文法分为4种类型，程序设计语言的大多数语法现象可用其中的（ ）描述。

- A. 上下文有关文法
- B. 上下文无关文法
- C. 正规文法
- D. 短语结构文法

第50题：运行下面的C程序代码段，会出现（ ）错误。

```
int k=0;
for(;k<100;);
{k++;}
```

- A. 变量未定义
- B. 静态语义
- C. 语法
- D. 动态语义

第51题：在数据库系统中，一般由DBA使用DBMS提供的授权功能为不同用户授权，其主要目的是为了保证数据库的（ ）。

- A. 正确性
- B. 安全性
- C. 一致性
- D. 完整性

第52题：给定关系模式R(U,F)，其中：U为关系模式R中的属性集，F是U上的一组函数依赖。假设U={A1, A2, A3, A4}，F={A1→A2, A1A2→A3, A1→A4, A2→A4}，那么关系R的主键应为（52）。函数依赖集F中的（53）是冗余的。

- A. A1
- B. A1A2
- C. A1A3
- D. A1A2A3

第53题：给定关系模式R(U,F)，其中：U为关系模式R中的属性集，F是U上的一组函数依赖。假设U={A1, A2, A3, A4}，F={A1→A2, A1A2→A3, A1→A4, A2→A4}，那么关系R的主键应为（52）。函数依赖集F中的（53）是冗余的。

- A. A1→A2
- B. A1A2→A3
- C. A1→A4
- D. A2→A4

第54题：给定关系R(A, B, C, D)和关系S(A, C, E, F)，对其进行自然连接运算R?S后的属性列为（54）个；与 $\sigma_{R.B=S.E}(R?S)$ 等价的关系代数表达式为（55）。

- A. 4
- B. 5
- C. 6
- D. 8

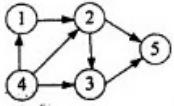
第55题：给定关系R(A, B, C, D)和关系S(A, C, E, F)，对其进行自然连接运算R?S后的属性列为（54）个；与 $\sigma_{R.B=S.E}(R?S)$ 等价的关系代数表达式为（55）。

- A. $\sigma_{2>7}(R \times S)$
- B. $\pi_{1,2,3,4,7,8}(\sigma_{1=5 \wedge 2>7 \wedge 3=6}(R \times S))$
- C. $\sigma_{2>7'}(R \times S)$
- D. $\pi_{1,2,3,4,7,8}(\sigma_{1=5 \wedge 2>'7' \wedge 3=6}(R \times S))$

第56题：下列查询B=“大数据”且F=“开发平台”，结果集属性列为A、B、C、F的关系代数表达式中，查询效率最高的是（ ）。

- A. $\pi_{1,2,3,8}(\sigma_{2='大数据' \wedge 1=5 \wedge 3=6 \wedge 8='开发平台'}(R \times S))$
- B. $\pi_{1,2,3,8}(\sigma_{1=5 \wedge 3=6 \wedge 8='开发平台'}(\sigma_{2='大数据'}(R) \times S))$
- C. $\pi_{1,2,3,8}(\sigma_{2='大数据' \wedge 1=5 \wedge 3=6}(R \times \sigma_{4='开发平台'}(S)))$
- D. $\pi_{1,2,3,8}(\sigma_{1=5 \wedge 3=6}(\sigma_{2='大数据'}(R) \times \sigma_{4='开发平台'}(S)))$

第57题：拓扑序列是有向无环图中所有顶点的一个线性序列，若有向图中存在弧 $\langle v, w \rangle$ 或存在从顶点 v 到 w 的路径，则在该有向图的任一拓扑序列中， v 一定在 w 之前。下面有向图的拓扑序列是（ ）。



- A. 41235
- B. 43125
- C. 42135
- D. 41325

第58题：设有一个包含 n 个元素的有序线性表。在等概率情况下删除其中的一个元素，若采用顺序存储结构，则平均需要移动（58）个元素；若采用单链表存储，则平均需要移动（59）个元素。

- A. 1
- B. $(n-1)/2$
- C. $\log n$
- D. n

第59题：设有一个包含 n 个元素的有序线性表。在等概率情况下删除其中的一个元素，若采用顺序存储结构，则平均需要移动（58）个元素；若采用单链表存储，则平均需要移动（59）个元素。

- A. 0
- B. 1
- C. $(n-1)/2$
- D. $n/2$

第60题：具有3个节点的二叉树有（ ）种形态。

- A. 2
- B. 3
- C. 5
- D. 7

第61题：以下关于二叉排序树（或二叉查找树、二叉搜索树）的叙述中，正确的是（ ）。

- A. 对二叉排序树进行先序、中序和后序遍历，都得到结点关键字的有序序列
- B. 含有 n 个结点的二叉排序树高度为 $(\log_2 n) + 1$
- C. 从根到任意一个叶子结点的路径上，结点的关键字呈现有序排列的特点
- D. 从左到右排列同层次的结点，其关键字呈现有序排列的特点

第62题：下表为某文件中字符的出现频率，采用霍夫曼编码对下列字符编码，则字符序列“bee”的编码为（62）；编码“110001001101”的对应的字符序列为（63）。

字符	a	b	c	d	e	f
频率(%)	45	13	12	16	9	5

- A. 10111011101
- B. 10111001100
- C. 001100100
- D. 110011011

第63题：下表为某文件中字符的出现频率，采用霍夫曼编码对下列字符编码，则字符序列“bee”的编码为（62）；编码“110001001101”的对应的字符序列为（63）。

字符	a	b	c	d	e	f
频率(%)	45	13	12	16	9	5

- A. bad
- B. bee
- C. face
- D. bace

第64题：两个矩阵 $A_{m \times n}$ 和 $B_{n \times p}$ 相乘，用基本的方法进行，则需要的乘法次数为 $m \times n \times p$ 。多个矩阵相乘满足结合律，不同的乘法顺序所需要的乘法次数不同。考虑采用动态规划方法确定 $M_i, M_{(i+1)}, \dots, M_j$ 多个矩阵连乘的最优顺序，即所需要的乘法次数最少。最少乘法次数用 $m[i, j]$ 表示，其递归式定义为：

$$m[i, j] = \begin{cases} 0 & i \geq j \\ \min_{i \leq k \leq j} \{m[i, k] + m[k+1, j] + p_i \cdot p_k \cdot p_j\} & i < j \end{cases}$$

其中 i, j 和 k 为矩阵下标，矩阵序列中 M_i 的维度为 $(p_{i-1}) \times p_i$ 采用自底向上的方法实现该算法来确定 n 个矩阵相乘的顺序，其时间复杂度为（64）。若四个矩阵 M_1, M_2, M_3, M_4 相乘的维度序列为2、6、3、10、3，采用上述算法求解，则乘法次数为

- (65)。
- A. $O(n^2)$
 - B. $O(n^2 \lg n)$
 - C. $O(n^3)$
 - D. $O(n^3 \lg n)$

第65题：两个矩阵 $A_{m \times n}$ 和 $B_{n \times p}$ 相乘，用基本的方法进行，则需要的乘法次数为 $m \times n \times p$ 。多个矩阵相乘满足结合律，不同的乘法顺序所需要的乘法次数不同。考虑采用动态规划方法确定 $M_i, M_{(i+1)}, \dots, M_j$ 多个矩阵连乘的最优顺序，即所需要的乘法次数最少。最少乘法次数用 $m[i, j]$ 表示，其递归式定义为：

$$m[i, j] = \begin{cases} 0 & i \geq j \\ \min_{i \leq k \leq j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & i < j \end{cases}$$

其中 i, j 和 k 为矩阵下标，矩阵序列中 M_i 的维度为 $(p_{i-1}) \times p_i$ 采用自底向上的方法实现该算法来确定 n 个矩阵相乘的顺序，其时间复杂度为(64)。若四个矩阵 M_1, M_2, M_3, M_4 相乘的维度序列为2、6、3、10、3，采用上述算法求解，则乘法次数为(65)。

- A. 156
- B. 144
- C. 180
- D. 360

第66题：以下协议中属于应用层协议的是(66)，该协议的报文封装在(67)。

- A. SNMP
- B. ARP
- C. ICMP
- D. X.25

第67题：以下协议中属于应用层协议的是(66)，该协议的报文封装在(67)。

- A. TCP
- B. IP
- C. UDP
- D. ICMP

第68题：某公司内部使用wb.xyz.com.cn作为访问某服务器的地址，其中wb是()。

- A. 主机名
- B. 协议名
- C. 目录名
- D. 文件名

第69题：如果路由器收到了多个路由协议转发的关于某个目标的多条路由，那么决定采用哪条路由的策略是()。

- A. 选择与自己路由协议相同的
- B. 选择路由费用最小的
- C. 比较各个路由的管理距离
- D. 比较各个路由协议的版本

第70题：与地址220.112.179.92匹配的路由表的表项是()。

- A. 220.112.145.32/22
- B. 220.112.145.64/22
- C. 220.112.147.64/22
- D. 220.112.177.64/22

第71题：Software entities are more complex for their size than perhaps any other human construct, because no two parts are alike (at least above the statement level). If they are, we make the two similar parts into one, a (71), open or closed. In this respect software systems differ profoundly from computers, buildings, or automobiles, where repeated elements abound. Digital computers are themselves more complex than most things people build; they have very large numbers of states. This makes conceiving, describing, and testing them hard. Software systems have orders of magnitude more (72) than computers do. Likewise, a scaling-up of a software entity is not merely a repetition of the same elements in larger size; it is necessarily an increase in the number of different elements. In most cases, the elements interact with each other in some (73) fashion, and the complexity of the whole increases much more than linearly.

The complexity of software is a(an) (74) property, not an accidental one. Hence descriptions of a software entity that abstract away its complexity often abstract away its essence. Mathematics and the physical sciences made great strides for three centuries by constructing simplified models of complex phenomena, deriving properties from the models, and verifying those properties experimentally. This worked because the complexities (75) in the models were not the essential properties of the phenomena. It does not work when the complexities are the essence. Many of the classical problems of developing software products derive from this essential complexity and its nonlinear increases with size. Not only technical problems but management problems as well come from the complexity.

- A. task
- B. job
- C. subroutine
- D. program

第72题: Software entities are more complex for their size than perhaps any other human construct, because no two parts are alike (at least above the statement level). If they are, we make the two similar parts into one, a (71), open or closed. In this respect software systems differ profoundly from computers, buildings, or automobiles, where repeated elements abound.

Digital computers are themselves more complex than most things people build; they have very large numbers of states. This makes conceiving, describing, and testing them hard. Software systems have orders of magnitude more (72) than computers do.

Likewise, a scaling-up of a software entity is not merely a repetition of the same elements in larger size; it is necessarily an increase in the number of different elements. In most cases, the elements interact with each other in some (73) fashion, and the complexity of the whole increases much more than linearly.

The complexity of software is a(an) (74) property, not an accidental one. Hence descriptions of a software entity that abstract away its complexity often abstract away its essence. Mathematics and the physical sciences made great strides for three centuries by constructing simplified models of complex phenomena, deriving properties from the models, and verifying those properties experimentally. This worked because the complexities (75) in the models were not the essential properties of the phenomena. It does not work when the complexities are the essence. Many of the classical problems of developing software products derive from this essential complexity and its nonlinear increases with size. Not only technical problems but management problems as well come from the complexity.

- A. states
- B. parts
- C. conditions
- D. expressions

第73题: Software entities are more complex for their size than perhaps any other human construct, because no two parts are alike (at least above the statement level). If they are, we make the two similar parts into one, a (71), open or closed. In this respect software systems differ profoundly from computers, buildings, or automobiles, where repeated elements abound.

Digital computers are themselves more complex than most things people build; they have very large numbers of states. This makes conceiving, describing, and testing them hard. Software systems have orders of magnitude more (72) than computers do.

Likewise, a scaling-up of a software entity is not merely a repetition of the same elements in larger size; it is necessarily an increase in the number of different elements. In most cases, the elements interact with each other in some (73) fashion, and the complexity of the whole increases much more than linearly.

The complexity of software is a(an) (74) property, not an accidental one. Hence descriptions of a software entity that abstract away its complexity often abstract away its essence. Mathematics and the physical sciences made great strides for three centuries by constructing simplified models of complex phenomena, deriving properties from the models, and verifying those properties experimentally. This worked because the complexities (75) in the models were not the essential properties of the phenomena. It does not work when the complexities are the essence. Many of the classical problems of developing software products derive from this essential complexity and its nonlinear increases with size. Not only technical problems but management problems as well come from the complexity.

- A. linear
- B. nonlinear
- C. parallel
- D. additive

第74题: Software entities are more complex for their size than perhaps any other human construct, because no two parts are alike (at least above the statement level). If they are, we make the two similar parts into one, a (71), open or closed. In this respect software systems differ profoundly from computers, buildings, or automobiles, where repeated elements abound.

Digital computers are themselves more complex than most things people build; they have very large numbers of states. This makes conceiving, describing, and testing them hard. Software systems have orders of magnitude more (72) than computers do.

Likewise, a scaling-up of a software entity is not merely a repetition of the same elements in larger size; it is necessarily an increase in the number of different elements. In most cases, the elements interact with each other in some (73) fashion, and the complexity of the whole increases much more than linearly.

The complexity of software is a(an) (74) property, not an accidental one. Hence descriptions of a software entity that abstract away its complexity often abstract away its essence. Mathematics and the physical sciences made great strides for three centuries by constructing simplified models of complex phenomena, deriving properties from the models, and verifying those properties experimentally. This worked because the complexities (75) in the models were not the essential properties of the phenomena. It does not work when the complexities are the essence. Many of the classical problems of developing software products derive from this essential complexity and its nonlinear increases with size. Not only technical problems but management problems as well come from the complexity.

- A. surface
- B. outside
- C. exterior
- D. essential

第75题: Software entities are more complex for their size than perhaps any other human construct, because no two parts are alike (at least above the statement level). If they are, we make the two similar parts into one, a (71), open or closed. In this respect software systems differ profoundly from computers, buildings, or automobiles, where repeated elements abound.

Digital computers are themselves more complex than most things people build; they have very large numbers of states. This makes conceiving, describing, and testing them hard. Software systems have orders of magnitude more (72) than computers do.

Likewise, a scaling-up of a software entity is not merely a repetition of the same elements in larger size; it is necessarily an increase in the number of different elements. In most cases, the elements interact with each other in some (73) fashion, and the complexity of the whole increases much more than linearly.

The complexity of software is a(an) (74) property, not an accidental one. Hence descriptions of a software entity that abstract away its complexity often abstract away its essence. Mathematics and the physical sciences made great strides for three centuries by constructing simplified models of complex phenomena, deriving properties from the models, and verifying those properties experimentally. This worked because the complexities (75) in the models were not the essential properties of the phenomena. It does not work when the complexities are the essence. Many of the classical problems of developing software products derive from this essential complexity and its nonlinear increases with size. Not only technical problems but management problems as well come from the complexity.

- A. fixed
- B. included
- C. ignored
- D. stabilized